perplexity

Can it be beat any tool harness benchmark

For beating harness we should create our own very atrong harness framework or tool or strategy
So what should be it and how we can do it

Yes—AgentBudget **can beat harness benchmarks**, but only if you build a harness whose advantage is measurable under constraints: higher success at equal or lower cost, fewer wasted steps, and better recovery from failures. You should not try to out-reason a frontier model; you should build the **control system around the model** that prevents the failure modes harnesses exist to solve. [1] [2]

The harness strategy

Call the core strategy:

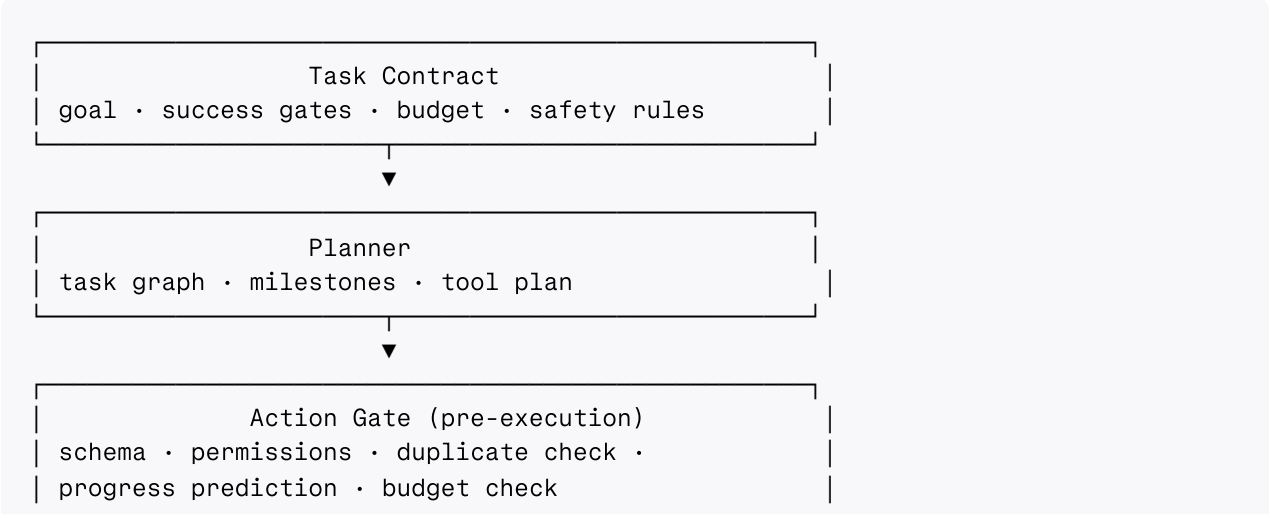
> **Constraint-Aware Self-Correcting Execution (CASE)**

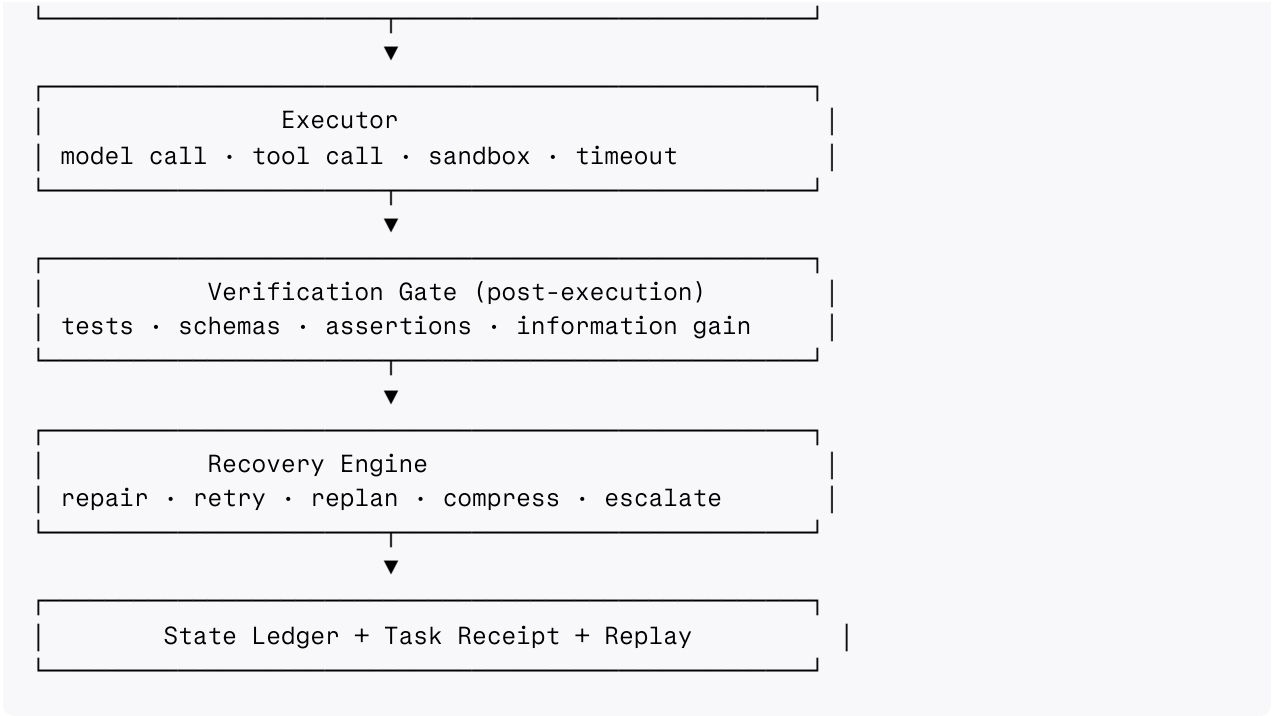
It means every agent step must answer four questions before continuing:

1. **Is this action allowed?**
2. **Is this action likely to make progress?**
3. **Did the previous action actually work?**
4. **Can we still finish within budget?**

Most harnesses only ask, “What tool should the model call next?” CASE asks whether that call is worth making.

Architecture: AgentBudget CASE Harness





This is stronger than a basic ReAct loop because it separates **deciding**, **acting**, **verifying**, and **recovering**. Production harness guidance emphasizes validation gates, checkpoints, recovery behavior, and escalation rules—not just model calls.[3] [1]

The five engines to build

1. Objective Verification Engine

Never let the model decide "I am done" by itself.

Use deterministic checks whenever possible:

- Run tests
- Compile code
- Validate JSON against a schema
- Check database state
- Verify file diffs
- Check API response fields
- Confirm a required artifact exists

Use an LLM judge only for subjective quality, and treat its score as evidence—not proof. Computational sensors such as linters, tests, and schema validators are fast and deterministic; inferential sensors such as LLM judges are useful but non-deterministic.[4]

Rule: every task must have at least one objective success gate.

2. Progress Sensor

Measure whether each action produces **information gain**:

```
progress_score =
  new_information
  + milestone_completion
  + error_reduction
  - repeated_action_penalty
```

If the same tool is called with nearly identical inputs and produces no downstream change, classify it as a loop. Tracking whether a call produced new data and forward progress is a practical way to detect non-convergence.[5] [6]

Example:

Action	Result	Progress score
Read `pricing.py`	New file content	+1
Read `pricing.py` again	Same content	-2
Run tests after edit	New failure details	+1
Run tests again without edit	Same failure	-2
Search web with same query	Same results	-2

3. Context Compaction Engine

Context exhaustion is one of the most common production agent failures, so compaction must be built in from the start rather than handled after overflow.[7] [4]

AgentBudget should maintain three layers:

- **Working context:** current goal, active file, latest test failure
- **Task memory:** completed milestones, decisions, learned constraints
- **Archive:** older tool outputs available on demand

When context approaches a limit, summarize completed work and remove resolved errors, duplicate tool output, and irrelevant retrieved chunks.

4. Tool Governor

The harness—not the model—should decide whether a tool call is safe and useful.

Before execution:

- Validate arguments against schema
- Check permissions
- Detect duplicate calls

- Check whether retrying is safe
- Estimate cost and expected value
- Block unsafe writes without approval

After execution:

- Capture errors
- Detect silent failures
- Record latency and output size
- Feed structured errors back to the model

Malformed calls should become corrective feedback, not crashes; repeated failures and circular edits should be treated as first-class loop signals. [6] [8]

5. Recovery Engine

Use a deliberate recovery policy:

Failure	Harness action
Transient network error	Retry with exponential backoff and jitter
Malformed tool arguments	Return a corrective error with valid schema
Same tool fails 3 times	Change strategy or escalate
Context near limit	Compact and checkpoint
No progress for N steps	Replan
Budget nearly exhausted	Produce best partial result
Unsafe side effect	Pause for human approval
Task complete but verification fails	Do not accept “done”

This prevents the two worst production behaviors: infinite loops and false “task complete” declarations. [8] [9]

What lets it beat benchmarks

A harness beats a benchmark when it converts wasted exploration into useful progress.

Example: coding-agent benchmark

Baseline agent behavior:

```
edit file → test fails → edit again → test fails →
edit same file again → test fails → claim done
```

AgentBudget CASE behavior:

```
edit file → test fails → extract failing assertion →
identify root cause → change approach → test passes
```

The second path may use fewer model calls and reach success more often. This is exactly the kind of improvement harness-level loop detection can create: one reported coding-agent harness improved Terminal Bench 2.0 performance from 52.8% to 66.5% without changing the underlying model, largely by preventing repeated edits to a broken approach. [10]

Benchmark to beat

Do **not** benchmark only "AgentBudget vs LangGraph."

Benchmark:

```
Same model
+ Same tools
+ Same tasks
+ Same hardware/API conditions
```

Then compare:

System	What it changes
Baseline scaffold	Minimal loop
Baseline + tracing	Observability only
Baseline + reflection	Model judges itself
AgentBudget CASE	Objective gates, progress sensing, recovery, budgets

Report a **Pareto frontier**:

```
X-axis: cost per successful task
Y-axis: task success rate
```

AgentBudget wins if it is above-left of baselines: same or better success at lower cost.

The most informative agent metric is not pass rate alone but **pass rate at a given cost**; evaluations should report accuracy versus cost and accuracy versus latency. [11]

Build order

v0.1: Beat the baseline

Build only:

1. Task contract
2. Tool registry with schemas

3. Budget controller
4. Duplicate-call detector
5. Test/schema verification gate
6. Retry policy
7. Task receipt

Target: **+10% success and -40% cost** versus a simple ReAct baseline.

v0.2: Beat reflection baselines

Add:

1. Progress sensor
2. Automatic replanning
3. Context compaction
4. Checkpoint/resume
5. Failure taxonomy

Target: **higher pass@k at lower cost** than naive reflection.

v0.3: Become a platform

Add:

1. LangGraph/CrewAI adapters
2. OpenTelemetry export
3. Policy engine
4. Dashboard
5. Public benchmark leaderboard

The winning claim

Do not claim:

> "AgentBudget is the best agent framework."

Claim:

> **"AgentBudget's CASE harness achieves higher task success at lower cost than baseline and reflection-only agent scaffolds by enforcing objective verification, progress-aware recovery, and budget-constrained execution."**

That is credible because it targets known harness failures—context rot, silent tool failures, loops, premature completion, and retry storms—with measurable controls. [9] [4] [8]

Your first public milestone should be:

> **AgentBudget CASE v0.1: A constraint-aware agent harness that improves task success while reducing cost per successful task on a reproducible benchmark.**

If you publish the benchmark, baseline code, and reproducible results, the AI community can verify the claim—and that is what turns a project into a milestone. [10] [11]

⁂

1. https://arize.com/blog/what-is-an-agent-harness/
2. https://rahulkashyap.dev/learn/agents-tools-harnesses/06-harness-failure-modes.html
3. https://plane.so/blog/what-is-an-agent-harness-and-how-does-it-work
4. https://www.fiddler.ai/blog/what-is-an-agent-harness
5. https://dev.to/agrawal_83a0b8e9e8b/the-7-ways-ai-agents-fail-in-production-and-how-to-catch-them-254a
6. https://dev.to/alexmercedcoder/designing-your-own-ai-harness-a-deep-dive-into-the-architecture-of-agent-loops-tools-context-2knl
7. https://harness-engineering.ai/blog/lessons-learned-from-deploying-ai-agents-in-production/
8. https://harness-engineering.ai/blog/agent-harness-complete-guide/
9. https://ashimsharma10.github.io/write-up/harness-context-loop-engineering/
10. https://x.com/RitOnchain/article/2072255424150241636
11. https://llmbook.icsgen-ai.org/part-6-agentic-ai/module-26-ai-agents/section-26.4.html
12. https://aclanthology.org/2026.findings-acl.86.pdf
13. https://kenrollins.github.io/gemma-forge/journal/architecture/01-reflexive-agent-harness-failure-modes/
14. https://arxiv.org/html/2607.15715v2
15. https://www.mongodb.com/company/blog/technical/agent-orchestration-tool-use-machinery-underneath
16. https://futureagi.com/blog/agent-harness-production-readiness/